

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras.layers import Dense, Conv2D, MaxPooling2D, Input, Flatten
from tensorflow.keras.models import Sequential
```

```
(x_train,y_train),(x_test,y_test) = tf.keras.datasets.mnist.load_data()
x_train=x_train.astype("float32")/255.0
x_test=x_test.astype("float32")/255.0
x_train.shape,x_test.shape
```

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-data11490434/11490434>  0s 0us/step
((60000, 28, 28), (10000, 28, 28))

```
x_train = np.expand_dims(x_train,-1)
x_test = np.expand_dims(x_test,-1)
x_train.shape, x_test.shape
```

((60000, 28, 28, 1), (10000, 28, 28, 1))

```
model = Sequential()
model.add(Input(shape=(28,28,1)))
model.add(Conv2D(32,(3,3),activation="relu",padding="same"))
model.add(MaxPooling2D(2,2))
model.add(Conv2D(64,(3,3),activation="relu",padding="same"))
model.add(MaxPooling2D(2,2))
model.add(Flatten())
model.add(Dense(64,activation="relu"))
model.add(Dense(10,activation="softmax"))
model.compile(optimizer="adam",loss="sparse_categorical_crossentropy",metric
history = model.fit(x_train,y_train,epochs=10,batch_size=256,validation_data
```

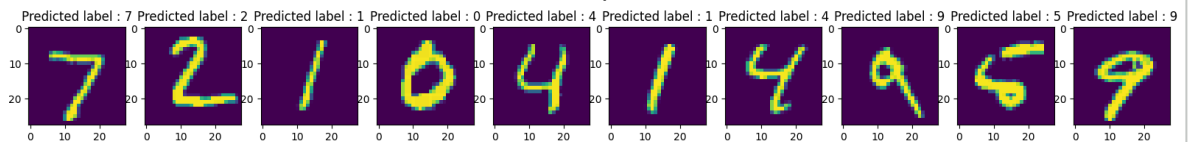
```
Epoch 1/10
235/235  9s 18ms/step - accuracy: 0.8084 - loss: 0.6730 -
Epoch 2/10
235/235  2s 7ms/step - accuracy: 0.9775 - loss: 0.0757 -
Epoch 3/10
235/235  2s 7ms/step - accuracy: 0.9854 - loss: 0.0498 -
Epoch 4/10
235/235  2s 8ms/step - accuracy: 0.9883 - loss: 0.0375 -
Epoch 5/10
235/235  2s 7ms/step - accuracy: 0.9908 - loss: 0.0295 -
Epoch 6/10
235/235  2s 7ms/step - accuracy: 0.9922 - loss: 0.0237 -
Epoch 7/10
235/235  2s 7ms/step - accuracy: 0.9939 - loss: 0.0192 -
Epoch 8/10
235/235  2s 7ms/step - accuracy: 0.9946 - loss: 0.0179 -
Epoch 9/10
235/235  2s 7ms/step - accuracy: 0.9946 - loss: 0.0166 -
```

Epoch 10/10

235/235 ————— 2s 7ms/step - accuracy: 0.9962 - loss: 0.0116 -

```
y_pred = model.predict(x_test)
n = 10
plt.figure(figsize=(20,4))
for i in range(n):
    ax = plt.subplot(2,n,i+1)
    plt.imshow(x_test[i])
    plt.title(f"Predicted label : {np.argmax(y_pred[i])}")
plt.show()
```

313/313 ————— 0s 1ms/step

Start coding or [generate](#) with AI.